

21-241 PCA Project

Sahiba Kaur, Tanay Mishra

December 2025

Abstract

Principal Component Analysis (PCA) is a standard dimensionality reduction technique that uses various topics from linear algebra. In this project we implemented PCA in python using two techniques from linear algebra:

1. the eigendecomposition of the sample covariance matrix
2. the singular value decomposition (SVD) of the centered data matrix.

We applied these implementations to a real-world diabetes dataset, and confirmed our understanding of linear algebra concepts by analyzing different results such as the variance, eigenvalues, and principal components.

1 Introduction

Principal Component Analysis (PCA) is a dimensionality reduction technique that aims to simplify complex data while retaining essential information. It transforms a set of correlated variables into a smaller number of uncorrelated variables called principal components, which effectively summarizes the data's key patterns.

PCA maximizes the variance of the projected data on a given axis while minimizing the reconstruction error. It identifies principal components, which are directions that capture the most significant variations in the data. Projecting data onto these principal components maximizes variance and minimizes reconstruction error, which gives the best possible lower dimension estimate of the data.

The covariance matrix is a square matrix where each element represents the covariance between two features. The diagonal elements represent the variances of individual features, and the other elements represent the covariances between different features.

This matrix is used to calculate eigenvectors and eigenvalues in PCA because it captures the variances and covariances between the features in a dataset. The principal components that PCA aims to identify are the directions of maximum variance in the data. These directions correspond to the eigenvectors of the covariance matrix, and the corresponding eigenvalues represent the magnitudes of variance along those directions. Projecting the data onto these PCs reduces dimensionality while preserving as much of the original information as possible.

2 Data

2.1 Dataset

Let X be the data matrix with shape $(n \times p)$. Then, we have

$$X = \begin{bmatrix} - & x_1^\top & - \\ - & x_2^\top & - \\ & \vdots & \\ - & x_n^\top & - \end{bmatrix},$$

where each row $x_i \in \mathbb{R}^p$ is one patient and each column is one feature.

We used the built-in diabetes dataset from scikit-learn. Each of the $n = 442$ rows is one patient, and there are $p = 10$ features per patient.

2.2 Centering the Data

PCA must be applied to centered data. This is because if we conduct the PCA analysis, we will be finding out what is the best way to reduce variance with respect to the origin. Instead, we want to figure out how to reduce variance with respect to the mean of the data, which is much more meaningful when analyzing data. Given the data matrix $X \in \mathbb{R}^{m \times n}$, where each row is an observation and each column is a feature, we compute the sample mean vector $\mu = \frac{1}{m} \sum_{i=1}^m x_i$ and then form the centered matrix $X_c = X - \mu^\top$. In our code, the function `center(X)` returns both X_c and μ .

3 SVD Method

When using SVD to do PCA, we can find the principal components easily through the right singular vectors of the centered data matrix.

First, the SVD must be computed. The methods for this are under section 6.9 of the lecture notes. The function we created for SVD returns U , Σ , and V^T . In our case, Σ was not stored as a matrix but rather as an array, as for all future calculations the only important part was the value of the singular values, not the dimension of Σ .

The `pca_via_svd(X, k)` function we made takes in a data matrix X and an integer k that represents the number of dimensions to reduce the data down to. The function first centers the data and then computes the SVD through both of the functions we made. After this, to find the k principal components, the function simply takes the first k columns of V^T that the SVD function returns. Finally, to project the data, the function multiplies the centered X matrix with the transpose of the components matrix. Projecting the data onto the selected principal components is what actually reduces the dimensionality and is an essential part of PCA.

4 Covariance method

For the covariance method, instead of finding an SVD, we can compute the sample covariance matrix to do the PCA.

The `pca_via_covariance(X, k)` function we made takes in the same arguments as the `pca_via_svd(X, k)` function, namely a data matrix X and an integer k that represents the number of dimensions to reduce the data down to. The function also centers the data using our `center(X)` function, just like `pca_via_svd`. Then, the function computes the sample covariance matrix. We implemented this according to page 208 of lecture notes. The sample covariance matrix S is

$$S = \frac{1}{m-1} X_c^T X_c$$

where X_c is the centered data matrix and m is from $X \in M_{m \times n}(\mathbb{R})$. The function then computes the eigenvalues and eigenvectors of S (and orders them in descending order), which are what allow us to extract the variance and conduct the PCA. To find the k principal components, we simply take the transpose of the first k eigenvectors. The projection process from there is the same as `pca_via_svd`.

5 Analysis of PCA

The following analyzes the results of PCA in the context of this real-world dataset and better conceptualizes what PCA tells us about the correlations in data.

5.1 Eigenvalues

Our code produced these eigenvalues (variance along each principal axis) Each eigenvalue λ_i measures the variance captured along the i th principal axis. The decrease in λ_i as i increases suggests that most of the meaningful variation is concentrated in the first few components.

i	λ_i
1	0.0091251944
2	0.0033839448
3	0.0027346174
4	0.0021666132
5	0.0015015451
6	0.0013667054
7	0.0012167022
8	0.0009834060
9	0.0001775964
10	0.0000194121

The amount of variance captured along a direction is given by the corresponding eigenvalue $\lambda_i = \frac{\sigma_i^2}{n-1}$ where σ_i is the i th singular value of A . (Here n was the rows in the data matrix, the number of patients).

5.2 Principal Axis Loadings

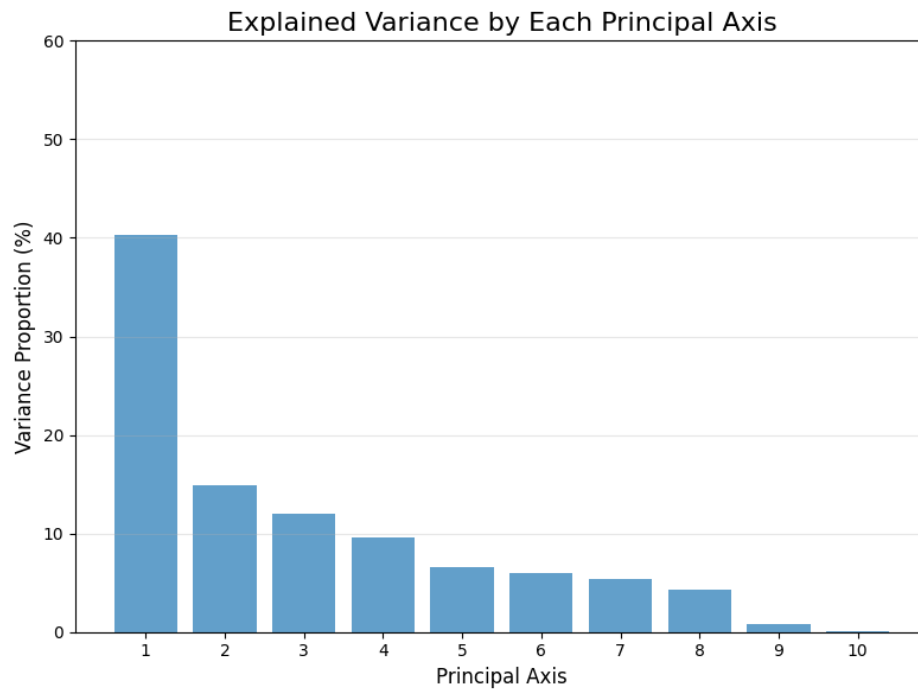
The feature `s1` has the largest magnitude, so it has the strongest influence on the first principal component. `bmi` and `bp` also have relatively large magnitudes. The signs tells us that `s1`, `bmi`, and `age` all contribute in the same direction on PC1 (their coefficients are negative), while `bp` and `s2` contribute in the opposite direction (their coefficients are positive). Overall, PC1 acts like a contrast between these two groups of features: patients with higher `s1`/BMI/age and lower `bp`/`s2` get large negative PC1 scores, while patients with the opposite results get positive PC1 scores.

Feature	Loading in v_1
age	-0.216
sex	-0.044
bmi	-0.495
bp	0.414
s1	-0.687
s2	0.226
s3	0.110
s4	-0.015
s5	-0.008
s6	-0.003

Now we will conceptually explain the linear algebra behind what is happening here. The matrix V_{meas} in our code comes from the SVD of the centered data matrix. The columns of V are the principal directions. Thus, the entries of a column v_i tell us how PC i is built as a linear combination of the other features. These coefficients are not the variance themselves, but they describe the direction in the feature space where the data varies the most.

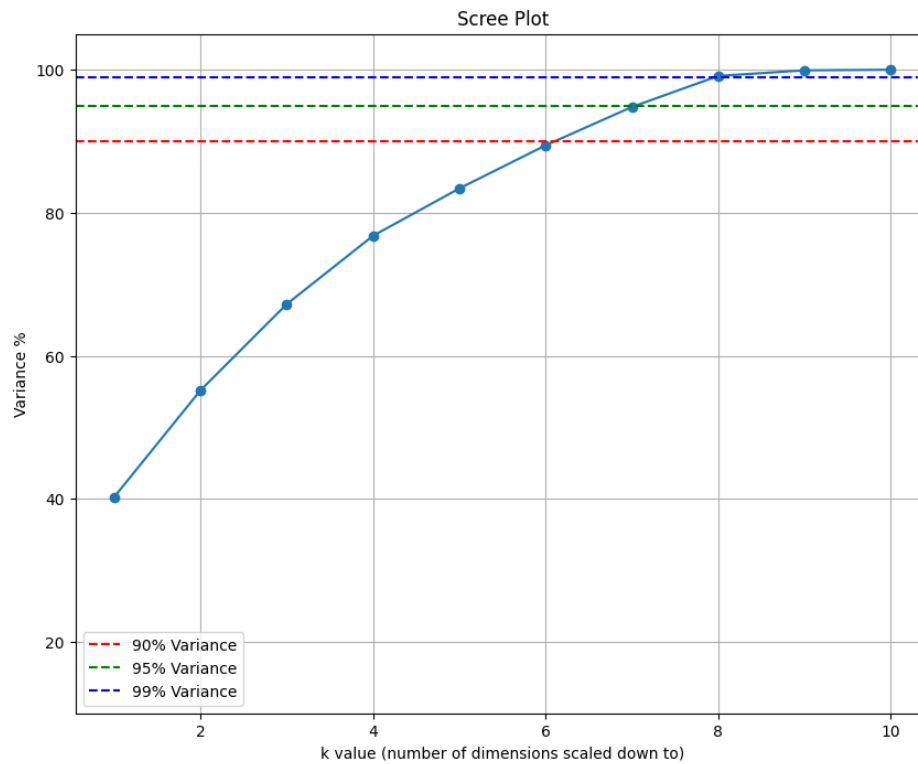
5.3 Explained Variance by Each Principal Axis

The bar chart shows how much of the dataset’s total variance is captured by each principal axis. In our code, we compute the variance ratio to represent explained variance percentages. This can be thought of as how much of the dataset’s “spread” we keep if summarizing the dataset using a particular principal component. As we have shown, the steep drop after a few PCs indicated that the dataset’s variation lies mainly in the first few components. This helps understand how PCA can reduce from 10 features to a smaller k while keeping most of the important information.



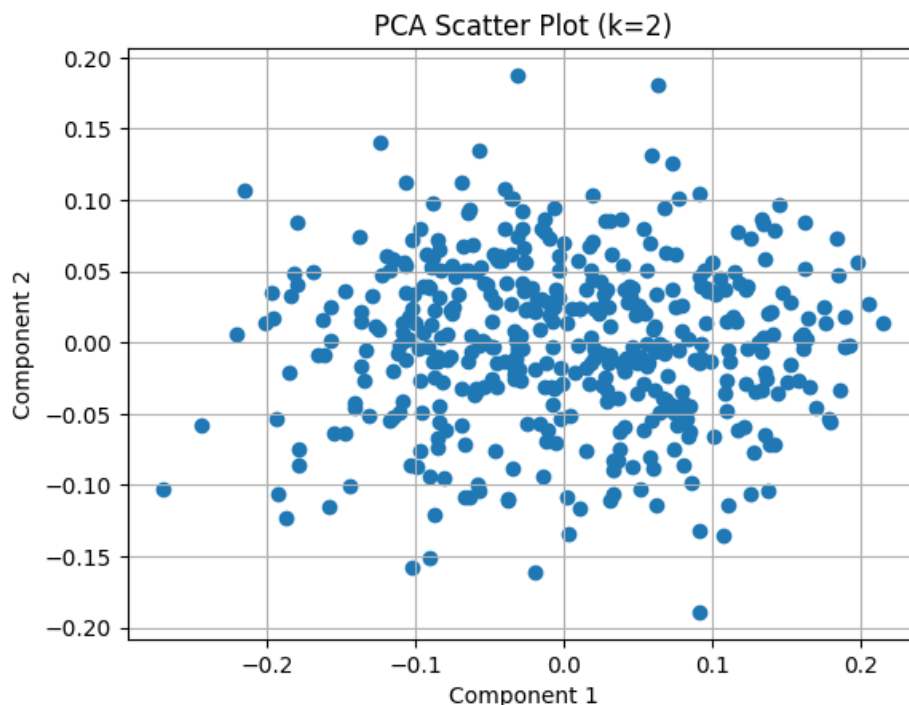
5.4 Scree Plot

The scree plot visualizes the cumulative variance explained as the number of retained principal components increases. To measure how much of the dataset's total variance is captured by the first k components, we compute the cumulative variance explained by the first k components. Specifically in our implementation, we compute `var_ratio = variances/total_variance` and `percent_variances = np.cumsum(var_ratio)`. For the diabetes data, two components explain about 55.15% of total variance, while six components explain about 89.43% and seven components explain about 94.79%. This indicates that reducing from 10 features to 6, 7 dimensions preserves most of the dataset's variance while still offering meaningful compression.



5.5 2D Scatter Plot

We projected the centered diabetes data onto $\mathbb{R}^2$ using the first two principal components. Let $A = U\Sigma V^\top$ be the SVD of the centered data. The first two principal axes are vectors v_1 and v_2 , which are the first two columns of V . Let $V_2 = [v_1, v_2]$. The projection is computed in code by `projected_X = centered_X @ k_components.T`. Each row of Y gives the (PC1, PC2) coordinates of one patient, which we plot to obtain the 2D scatter plot.



5.6 Comparing PCA via SVD and Covariance

5.7 Orthonormality of Principal Components

The principal axes form an orthonormal set. This comes from the fact that the covariance matrix S is symmetric, so it has an orthonormal eigenbasis. In SVD, V is an orthogonal matrix, so each principal axis is perpendicular to the others and has length 1. Our code verifies this property using `has_orthonormal_columns`. When we test $k = 10$, the `k_components` is essentially $V^\top$, so checking $(V^\top)^\top(V^\top) = VV^\top = I$ confirms orthonormality. We also check that the projected matrix has orthogonal columns using `has_orthogonal_columns(projected_X)`. This matches the SVD identity $AV = U\Sigma$ whose columns are orthogonal because U is orthogonal and Σ only scaled either direction.

6 Reconstruction and Error

Let $V_k = [v_1 \cdots v_k]$ and $Y = AV_k$. Our lecture notes (page 214) state that the rank- k PCA reconstruction of the centered data is $\tilde{A} = AV_k V_k^\top$ and for each data point, $\tilde{x}_i = V_k y_i + \mu$.

This is exactly what our code computes at `reconstructed = projected @ k_components`. Here, `projected` is Y and `k_components` stores $V_k^\top$.

6.1 Deriving the Formula $Y = AV_k$

Let $x \in \mathbb{R}^{m \times n}$ be the data matrix with m observations and n features. After centering, we obtain $A = X - \mu^\top$, so the i th centered observation is the row vector $\hat{x}_i$ of A .

Let $v_1, \dots, v_k \in \mathbb{R}^n$ be the first k principal directions and define $V_k = [v_1 \cdots v_k] \in \mathbb{R}^{n \times k}$.

For a single centered data point $\hat{x}_i$, its coordinate along directions v_j is the scalar $v_j^\top \hat{x}_i$. Thus, the vector of the first k PCA coordinates/scores for $\hat{x}_i$ is

$$y_i = \begin{bmatrix} v_1^\top \hat{x}_i \\ \vdots \\ v_k^\top \hat{x}_i \end{bmatrix} = V_k^\top \hat{x}_i \in \mathbb{R}^k$$

Consider this vector y_i as the compressed representation of observation i . We now stack all of these coordinate vectors into a matrix

$$Y = \begin{bmatrix} y_1^\top \\ \vdots \\ y_m^\top \end{bmatrix} \in \mathbb{R}^{m \times k}$$

Thus, row i of Y contains the k PCA coordinates of the i th observation. Column j of Y contains the scores along the j -th principal direction. Consider entry (i, j) of product AV_k . Row i of A is $\hat{x}_i$ and column j of V_k is v_j , so

$$(AV_k)_{ij} = \hat{x}_i^\top v_j = v_j^\top \hat{x}_i$$

This is exactly the definition of the j th PCA coordinate of the i th observation. Therefore, the i th rows of AV_k is $y_i^\top$, and we conclude that

$$Y = AV_k$$

Essentially, to get PCA coordinates, we center the data so the mean is at the origin, then take dot products with the first k principal directions, and then store those dot products as new coordinates. The formula $Y = AV_k$ is the matrix version of doing these dot products in one step.

7 Bibliography

Kovalev, P. (2025). 21-241: Matrices and Linear Transformations – Lecture notes, Fall 2025. Carnegie Mellon University

Data is from sklearn diabetes dataset, derived from <https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html> and python sklearn library

A PCA Implementation (Full Code)

In the attached jupyter notebook in the pensieve submission. We have organized our code so it is easy to view the specific sections and figures corresponding to it, as well as left comments to explain our thought process.

```
# -*- coding: utf-8 -*-
"""PCA 21241 Project.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1Nc_c1X8hirLXY1eucxMWDJ2GJv7YQ-H9

# PCA Final Project
Sahiba Kaur & Tanay Mishra

# Creating All Necessary Functions
"""

#implementing PCA based on Pr. Pavel and youtube
import numpy as np

#all methods are according to notes page 206-218:
def do_sortof_svd(A):
    #first need to get A transpose A
    AT = np.transpose(A)
    ATA = AT @ A
    #need to find eigenvalues and eigenvectors
    eigenvalues, eigenvectors = np.linalg.eigh(ATA) #eigh sorts in ascending order
    #need to flip the order of these for both eigenvectors and eigenvalues

    reverse_order = np.argsort(eigenvalues)[::-1]
    #to apply reverse order:
    eigenvalues = eigenvalues[reverse_order]
    eigenvectors = eigenvectors[:, reverse_order] #this should be equal to V in
    ↪ svd U(Sigma)Vt
    # now both go from Largest to Smallest eigenvalues and corresponding
    ↪ eigenvectors
```

```

sqrt_eigenvalues = np.sqrt(eigenvalues) #we dont need this in a matrix since
↳ we only need the values

#U is found page 181 lecture notes but basically  $u_i = (A*v_i)/o_i$  where  $o_i$ 
↳ is  $\sqrt{\lambda_i}$ 
U = A @ eigenvectors
for i in range(len(sqrt_eigenvalues)):
    U[:, i] /= sqrt_eigenvalues[i]

# these are U, Sigma, and  $V^T$ 
return U, sqrt_eigenvalues, np.transpose(eigenvectors)

def center(X):
    #centering done by following page 207 of lecture notes
    m = X.shape[0] #this gets the number of data points from the matrix X
    mean_vector = (1/m) * np.sum(X, axis=0) #should sum cols
    centered_X = X - mean_vector
    return centered_X, mean_vector

def pca_via_svd(X, k):
    #here k is the number of best fit things so we can do the error percent stuff
    ↳ maybe

    centered_X, mean_vector = center(X)
    U, values, VT = do_sortof_svd(centered_X)

    k_components = VT[:k]
    projected_X = centered_X @ np.transpose(k_components) #projects data onto new
    ↳ k space
    sigma_squared = (values[:k]**2) #this gets the first k singular values and
    ↳ squares them

    return projected_X, sigma_squared, k_components, VT

def pca_via_covariance(X, k):
    m_data, n_features = X.shape
    centered_X, mean_vector = center(X)
    #this below is the sample covariance matrix page 208
    S = (1 / (m_data - 1)) * (centered_X.T @ centered_X)

    #steps below are same as some of the svd stuff
    eigenvalues, eigenvectors = np.linalg.eigh(S)
    reverse_order = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[reverse_order]
    eigenvectors = eigenvectors[:, reverse_order]

```

```

    k_components = eigenvectors[:, :k].T

    projected_X = centered_X @ k_components.T

    explained_variance = eigenvalues[:k]

    return projected_X, explained_variance, k_components

"""# Using the PCA Functions

Getting data from sklearn diabetes dataset \
There are 10 features, named in feature_names \
Printing out important data such as
*   Eigenvalues of the sample covariance matrix (variance along given direction)
*   The first 3 principal axes in terms of the original variables
*   The variance given each k value (number of components)

"""

import matplotlib.pyplot as plt

import pandas as pd

from sklearn import datasets
diabetes = datasets.load_diabetes()
X = diabetes.data

m_data, n_features = X.shape
feature_names = ["age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6"]

projected, variances, k_components, V_meas = pca_via_svd(X, 10)
centered_X, mean_vector = center(X)

eigenvalues = (variances) / (m_data - 1)
for i, lam in enumerate(eigenvalues):
    print(f"    _{i} = {lam}")

total_variance = np.sum(variances)
var_ratio = variances / total_variance
percent_variances = np.cumsum(var_ratio)

print("\n")
print("="*50)

```

```

print("\nFirst principal axis (v):")
for i, name in enumerate(feature_names):
    print(f" {name:12s}: {V_meas[i, 0]:8.5f}")

print("\nSecond principal axis (v):")
for i, name in enumerate(feature_names):
    print(f" {name:12s}: {V_meas[i, 1]:8.5f}")

print("\nThird principal axis (v):")
for i, name in enumerate(feature_names):
    print(f" {name:12s}: {V_meas[i, 2]:8.5f}")

print("\n")
print("="*50)
print("\n")

U, eigen, VT = do_sortof_svd(X)

for i in range(1, 11):
    projected, variances, k_components, VT = pca_via_svd(X, i)
    i_variance = np.sum(variances)
    # print(variances)
    i_ratio = i_variance / total_variance
    print(f"variance with {i} components: {i_ratio*100} %")

print("\n")

r_m = len(eigen)

"""# Plotting the Diabetes Data

1. Variance Explained by Each Principal Axis
2. Scree plot showing variance % for each k components
3. Projecting the data onto R2 with k = 2
"""

fig, ax = plt.subplots(figsize=(8, 6))
ax.bar(range(1, r_m+1), 100 * var_ratio, alpha=0.7)
ax.set_xlabel('Principal Axis', fontsize=12)
ax.set_ylabel('Variance Proportion (%)', fontsize=12)
ax.set_title('Explained Variance by Each Principal Axis', fontsize=16)
ax.set_xticks(range(1, r_m+1))
ax.set_ylim(0, 60)
ax.grid(True, axis='y', alpha=0.3)

```

```

plt.tight_layout()
plt.show()

plt.figure(figsize=(10,8))
plt.plot(range(1, len(percent_variances)+1), percent_variances*100, marker='o')
plt.axhline(90, color='r', linestyle='--', label='90% Variance')
plt.axhline(95, color='g', linestyle='--', label='95% Variance')
plt.axhline(99, color='b', linestyle='--', label='99% Variance')
plt.xlabel('k value (number of dimensions scaled down to)')
plt.ylabel('Variance %')
plt.title('Scree Plot')
plt.ylim(10, 105)
plt.legend()
plt.grid(True)
plt.show()

projected_X, variances, k_components, last = pca_via_svd(X, 2)

plt.scatter(projected_X[:, 0], projected_X[:, 1])

plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.title("PCA Scatter Plot (k=2)")
plt.grid(True)
plt.show()

"""# Reconstructing the Original Data"""

diabetes = datasets.load_diabetes()
X = diabetes.data

chosen_k = 7

m_data, n_features = X.shape
centered_X, mean_vector = center(X)
projected, variances, k_components, VT = pca_via_svd(X, chosen_k)

reconstructed = projected @ k_components
print("\nOriginal data ")
print(centered_X)
print("\nReconstructed data from k =", chosen_k)
print(reconstructed)

relative_error = np.linalg.norm(centered_X - reconstructed, 'fro') /
↪ np.linalg.norm(centered_X, 'fro')

```

```

print(f"\noriginal dimension:    {n_features}")
print(f"k_value:                {chosen_k}")
print(f"Relative error:          {100*relative_error:.2f}%")

print("Different errors for different k values\nOriginal dimension is 10")
for k_i in range (1, 11):
    projected_temp, _, k_components_temp, _ = pca_via_svd(X, k_i)

    reconstructed_temp = projected_temp @ k_components_temp

    relative_error_temp = np.linalg.norm(centered_X - reconstructed_temp, 'fro') /
    ↪ np.linalg.norm(centered_X, 'fro')
    print(f"\nk_value:          {k_i}")
    print(f"Relative error:    {100*relative_error_temp:.2f}%")

reconstructed_original_data = reconstructed + mean_vector
print("Original Data:\n")
print(X)
print("\n\nReconstructed Data from k = 7:\n")
print(reconstructed_original_data)

"""# Checking Mean Squared Reconstructed Error
computing mean squared reconstruction error per observation for several values
↪ of k and comparing to the theoretical variance (which is the sum of the
↪ eigenvalues from k onwards)
"""

def getting_msre(X, k):
    proj, _, components, _ = pca_via_svd(X, k)
    m_data, n_features = X.shape
    centered_X, mean_vector = center(X)

    X_reconstructed = proj @ components + mean_vector

    #this should be the actual mean squared error per observation
    msre = np.mean(np.sum((X - X_reconstructed)**2, axis=1))

    return msre

_, eigenvals, _ = pca_via_covariance(X, 10)

for i in range (1, 10):
    msre = getting_msre(X, i)
    theoretical_error = np.sum(eigenvals[i:])

    close = ((msre - theoretical_error) / theoretical_error) < 0.01
    print(f"Is the MSRE equal to the theoretical for k = {i}: {close}")

```

```

"""# Comparing PCA via SVD and Covariance
If the methods achieve the same result, the outputted matrix will have all True
"""

diabetes = datasets.load_diabetes()
X = diabetes.data

chosen_k = 10

m_data, n_features = X.shape
centered_X, mean_vector = center(X)

projected_via_svd, _, k_components_svd, _ = pca_via_svd(X, chosen_k)
projected_via_covariance, _, k_components_covariance = pca_via_covariance(X,
    ↪ chosen_k)

print(f"Are the projections the same: \n{np.isclose(projected_via_svd,
    ↪ projected_via_covariance)}")
print(f"\n\nAre all the components the same: \n{np.isclose(k_components_svd,
    ↪ k_components_covariance)}")

"""# Orthonormality of Principal Components
Verifying that principal components are orthonormal and that
projecting onto them produces an orthogonal basis
"""

def has_orthonormal_columns(A):
    #checking if  $(A^T)A = I$ 
    ATA = A.T @ A
    Identity = np.eye(A.shape[1])
    return np.allclose(ATA, Identity)

def has_orthogonal_columns(A):
    #checking if  $(A^T)A$  is diagonal
    ATA = A.T @ A
    return np.allclose(ATA, np.diag(np.diag(ATA)))

projection_to_check, _, k_components_to_check, _ = pca_via_svd(X, 10)

print(f"Are the principal components orthonormal:
    ↪ {has_orthonormal_columns(k_components_to_check)}")
print(f"Is the projected basis orthogonal:
    ↪ {has_orthogonal_columns(projection_to_check)}")

```